

Mediator and CQRS Design Patterns

This chapter covers the building blocks of the next chapter, which is about **Vertical Slice Architecture**. We begin with a quick overview of Vertical Slice Architecture to give you an idea of the end goal. Then, we explore the **Mediator** design pattern, which plays the role of the middle-man between the components of our application. That leads us to the **Command Query Responsibility Segregation (CQRS)** pattern, which describes how to divide our logic into commands and queries. Finally, to piece all of that together, we explore MediatR, an open source implementation of the Mediator design pattern.

The following topics are covered in this chapter:

- A high-level overview of Vertical Slice Architecture
- Implementing the Mediator pattern
- Implementing the CQRS pattern
- Using MediatR as a mediator

Let's begin with the end goal.

A high-level overview of Vertical Slice Architecture

Before starting, let's look at the end goal of this chapter and the next. This way, it should be easier to follow the progress toward that goal throughout the chapter.

As we covered in *Chapter 12, Understanding Layering*, a layer groups classes together based on shared responsibilities. So, classes containing data access code are part of the data access layer (or infrastructure). In diagrams, layers are usually represented using horizontal slices, like this:

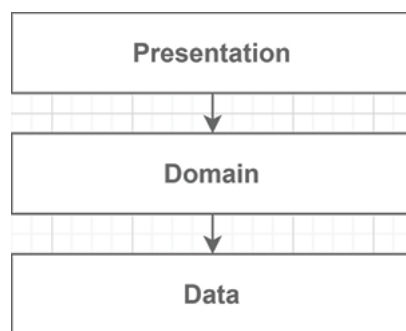


Figure 14.1: Diagram representing layers as horizontal slices

The “vertical slice” in “Vertical Slice Architecture” comes from that; a vertical slice represents the part of each layer that creates a specific feature. So, instead of dividing the application into layers, we divide it by feature. A feature manages its data access code, its domain logic, and possibly even its presentation code. We are decoupling the features from one another by doing this but keeping each feature’s components close together. When we add, update, or remove a feature using layering, we change one

or more layers. Unfortunately, “one or more layers” too often translates to “all layers.”

On the other hand, with vertical slices, keeping features in isolation allows us to design them independently instead. From a layering perspective, it's like flipping your way of thinking about software to a 90° angle:

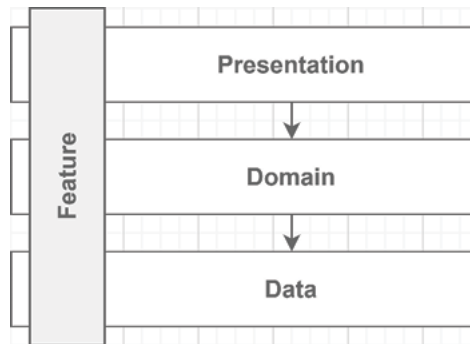


Figure 14.2: Diagram representing a vertical slice crossing all layers

Vertical Slice Architecture does not dictate the use of **CQRS**, the **Mediator** pattern, or **MediatR**, but these tools and patterns flow very well together, as we see in the next chapter. Nonetheless, these are just tools and patterns that you can use or change in your implementation using different techniques; it does not matter and does not change the concept.

The goal is to encapsulate features together, use CQRS to divide the application into requests (commands and queries), and use MediatR as the mediator of that CQRS pipeline, decoupling the pieces from one another.

You now know the plan—we will explore Vertical Slice Architecture later; meanwhile, let's start with the Mediator design pattern.

Implementing the Mediator pattern

The **Mediator** pattern is another GoF design pattern that controls how objects interact with one another (making it a behavioral pattern).

Goal

The mediator's role is to manage the communication between objects (colleagues). Those colleagues should not communicate together directly but use the mediator instead. The mediator helps break tight coupling between these colleagues.

A mediator is a middleman who relays messages between colleagues.

Design

Let's start with some UML diagrams. From a very high level, the Mediator pattern is composed of a mediator and colleagues:

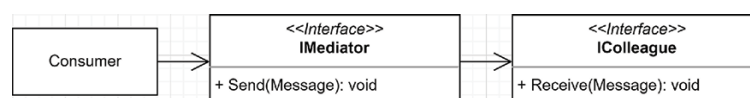


Figure 14.3: Class diagram representing the Mediator pattern

When an object in the system wants to send a message to one or more colleagues, it uses the mediator. Here is an example of how it works:

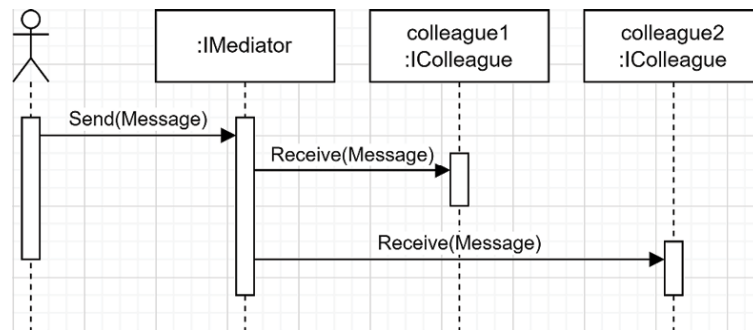


Figure 14.4: Sequence diagram of a mediator relaying messages to colleagues

That is also valid for colleagues; if they need to talk to each other, a colleague must also use the mediator. We could update the diagram as follows:

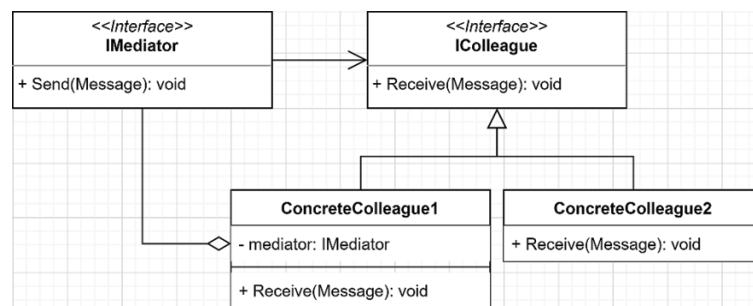


Figure 14.5: Class diagram representing the Mediator pattern including colleagues' collaboration

In that diagram, `ConcreteColleague1` is a colleague but also the consumer of the mediator. For example, that colleague could send a message to another colleague using the mediator, like this:

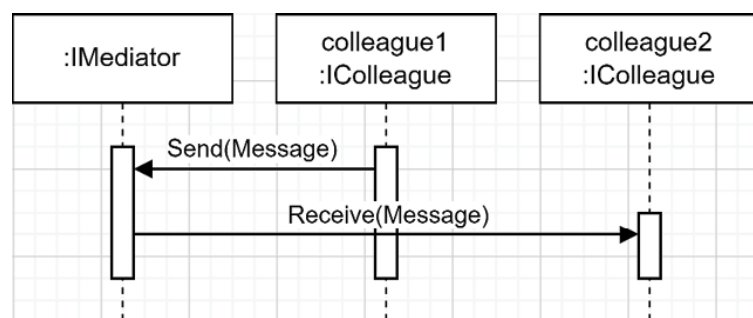


Figure 14.6: Sequence diagram representing colleague1 communicating with colleague2 through the mediator

From a mediator standpoint, its implementation will most likely contain a collection of colleagues to communicate with, like this:

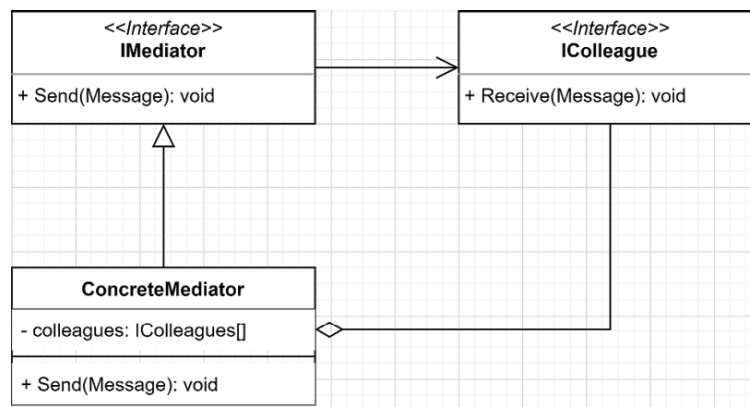


Figure 14.7: Class diagram representing a simple hypothetical concrete mediator implementation

All of those UML diagrams are useful, but enough of that; it is now time to look at some code.

Project – Mediator (IMediator)

The Mediator project consists of a simplified chat system using the Mediator pattern. Let's start with the interfaces:

```

namespace Mediator;
public interface IMediator
{
    void Send(Message message);
}
public interface IColleague
{
    string Name { get; }
    void ReceiveMessage(Message message);
}
public class Message
{
    public Message(IColleague from, string content)
    {
        Sender = from ?? throw new ArgumentNullException(nameof(from));
        Content = content ?? throw new ArgumentNullException(nameof(content));
    }
    public IColleague Sender { get; }
    public string Content { get; }
}
  
```

The system is composed of the following:

- `IMediator`, which sends messages.
- `IColleague`, which receives messages and has a `Name` property (to output something).
- The `Message` class, which represents a message sent by an `IColleague` implementation.

Now to the implementation of the `IMediator` interface.

`ConcreteMediator` broadcasts the messages to all `IColleague` instances without discrimination:

```

public class ConcreteMediator : IMediator
{
  
```

```

private readonly List<IColleague> _colleagues;
public ConcreteMediator(params IColleague[] colleagues)
{
    ArgumentNullException.ThrowIfNull(colleagues);
    _colleagues = new List<IColleague>(colleagues);
}
public void Send(Message message)
{
    foreach (var colleague in _colleagues)
    {
        colleague.ReceiveMessage(message);
    }
}
}

```

That mediator is simple; it forwards all the messages it receives to every colleague it knows. The last part of the pattern is `ConcreteColleague`, which delegates the messages to an `IMessageWriter<TMessage>` interface:

```

public class ConcreteColleague : IColleague
{
    private readonly IMessageWriter<Message> _messageWriter;
    public ConcreteColleague(string name, IMessageWriter<Message> messageWriter)
    {
        Name = name ?? throw new ArgumentNullException(nameof(name));
        _messageWriter = messageWriter ?? throw new ArgumentNullException(nameof(messageWriter));
    }
    public string Name { get; }
    public void ReceiveMessage(Message message)
    {
        _messageWriter.Write(message);
    }
}

```

That class could hardly be simpler: it takes a name and an `IMessageWriter<TMessage>` implementation when created, then it stores a reference for future use.

The `IMessageWriter<TMessage>` interface serves as a presenter to control how the messages are displayed and has nothing to do with the Mediator pattern. Nevertheless, it is an excellent way to manage how a `ConcreteColleague` object handles messages. Here is the code:

```

namespace Mediator;
public interface IMessageWriter<TMessage>
{
    void Write(TMessage message);
}

```

Let's use that chat system now. The consumer of the system is the integration test defined in the `MediatorTest` class:

```

public class MediatorTest
{
    [Fact]
    public void Send_a_message_to_all_colleagues()
    {
        // Arrange
        var (millerWriter, miller) = CreateConcreteColleague("Miller");
    }
}

```

```

var (orazioWriter, orazio) = CreateConcreteColleague("Orazio");
var (fletcherWriter, fletcher) = CreateConcreteColleague("Fletcher");

```

The test starts by defining three colleagues with their own `TestMessageWriter` implementation (names were randomly generated).

```

var mediator = new ConcreteMediator(miller, orazio, fletcher);
var expectedOutput = @"[Miller]: Hey everyone!
[Orazio]: What's up Miller?
[Fletcher]: Hey Miller!
";

```

In the second part of the preceding `Arrange` block, we create the subject under test (`mediator`) and register the three colleagues. At the end of that `Arrange` block, we also define the expected output of our test. It is important to note that we control the output from the `TestMessageWriter` implementation (defined at the end of the `MediatorTest` class).

```

// Act
mediator.Send(new Message(
    from: miller,
    content: "Hey everyone!"
));
mediator.Send(new Message(
    from: orazio,
    content: "What's up Miller?"
));
mediator.Send(new Message(
    from: fletcher,
    content: "Hey Miller!"
));

```

In the preceding `Act` block, we send three messages through `mediator`, in the expected order.

```

// Assert
Assert.Equal(expectedOutput, millerWriter.Output.ToString());
Assert.Equal(expectedOutput, orazioWriter.Output.ToString());
Assert.Equal(expectedOutput, fletcherWriter.Output.ToString());
}

```

In the `Assert` block, we ensure that all colleagues received the messages.

```

private (TestMessageWriter, ConcreteColleague) CreateConcreteColleague(string name)
{
    var messageWriter = new TestMessageWriter();
    var concreteColleague = new ConcreteColleague(name, messageWriter);
    return (messageWriter, concreteColleague);
}

```

The `CreateConcreteColleague` method is a helper method that encapsulates the creation of the colleagues, enabling us to write the one-liner declaration used in the `Arrange` section of the test.

```

private class TestMessageWriter : IMessageWriter<Message>
{
    public StringBuilder Output { get; } = new StringBuilder();
    public void Write(Message message)
    {
        Output.AppendLine($"{message.Sender.Name}: {message.Content}");
    }
}

} // Closing the MediatorTest class

```

Finally, the `TestMessageWriter` class writes the messages into `StringBuilder`, making it easy to assert the output. If we were to build a GUI for that, we could write an implementation of `IMessageWriter<Message>` that writes to that GUI; in the case of a web UI, it could be using **SignalR**, for example.

To summarize the sample: the consumer (the unit test) sent messages to the colleagues through the mediator. Those messages were written in the `StringBuilder` instance of each `TestMessageWriter`. Finally, we asserted that all colleagues received the expected messages. That illustrates that using the Mediator pattern allowed us to break the direct coupling between the colleagues; the messages reached them without them knowing about each others’.

In theory, colleagues should communicate through the mediator, so the Mediator pattern would not be complete without that. Let’s implement a chatroom to tackle that concept.

Project – Mediator (IChatRoom)

In the last code sample, the classes were named after the Mediator pattern actors, as shown in the diagram of *Figure 14.7*. While this example is very similar, it uses domain-specific names instead and implements a few more methods to manage the system showing a more tangible implementation. Let’s start with the abstractions:

```

namespace Mediator;
public interface IChatRoom
{
    void Join(IParticipant participant);
    void Send(ChatMessage message);
}

```

The `IChatRoom` interface is the mediator and it defines two methods instead of one:

- `Join`, which allows `IParticipant` to join `IChatRoom`.
- `Send`, which sends a message to the others.

```

public interface IParticipant
{
    string Name { get; }
    void Send(string message);
    void ReceiveMessage(ChatMessage message);
    void ChatRoomJoined(IChatRoom chatRoom);
}

```

The `IParticipant` interface also has a few more methods:

- `Send`, to send messages.
- `ReceiveMessage`, to receive messages from the other `IParticipant` objects.
- `ChatRoomJoined`, to confirm that the `IParticipant` object has successfully joined a chatroom.

```
public class ChatMessage
{
    public ChatMessage(IParticipant from, string content)
    {
        Sender = from ?? throw new ArgumentNullException(nameof(from));
        Content = content ?? throw new ArgumentNullException(nameof(content));
    }
    public IParticipant Sender { get; }
    public string Content { get; }
}
```

`ChatMessage` is the same as the previous `Message` class, but it references `IParticipant` instead of `IColleague`.

Let's now look at the `IParticipant` implementation:

```
public class User : IParticipant
{
    private IChatRoom? _chatRoom;
    private readonly IMessageWriter<ChatMessage> _messageWriter;
    public User(IMessageWriter<ChatMessage> messageWriter, string name)
    {
        _messageWriter = messageWriter ?? throw new ArgumentNullException(nameof(messageWriter));
        Name = name ?? throw new ArgumentNullException(nameof(name));
    }
    public string Name { get; }
    public void ChatRoomJoined(IChatRoom chatRoom)
    {
        _chatRoom = chatRoom;
    }
    public void ReceiveMessage(ChatMessage message)
    {
        _messageWriter.Write(message);
    }
    public void Send(string message)
    {
        if (_chatRoom == null)
        {
            throw new ChatRoomNotJoinedException();
        }
        _chatRoom.Send(new ChatMessage(this, message));
    }
}

public class ChatRoomNotJoinedException : Exception
{
    public ChatRoomNotJoinedException()
        : base("You must join a chat room before sending a message.") { }
}
```

The `User` class represents our default `IParticipant`. A `User` instance can chat in only one `IChatRoom`; that is set when calling the `ChatRoomJoined` method. When it receives a message, it delegates it to its `IMessageWriter<ChatMessage>`. Finally, a `User` instance can send

a message by delegating it to the mediator (`IChatRoom`). The `Send` method throws a `ChatRoomNotJoinedException` to enforce that the `User` has joined a chat room before sending messages (code-wise: the `_chatRoom` field is nullable).

We could create a `Moderator`, `Administrator`, `SystemAlerts`, or any other `IParticipant` implementation as we see fit, but not in this sample. I am leaving that to you to experiment with the Mediator pattern.

Now to the `IChatRoom` implementation:

```
public class ChatRoom : IChatRoom
{
    private readonly List<IParticipant> _participants = new List<IParticipant>();
    public void Join(IParticipant participant)
    {
        _participants.Add(participant);
        participant.ChatRoomJoined(this);
        Send(new ChatMessage(participant, "Has joined the channel"));
    }
    public void Send(ChatMessage message)
    {
        _participants.ForEach(participant => participant.ReceiveMessage(message));
    }
}
```

`ChatRoom` is even slimmer than `User`; it allows `IParticipant` to join in and sends `ChatMessage` to all registered participants. When joining a `ChatRoom`, it keeps a reference on that `IParticipant`, tells that `IParticipant` that it has successfully joined, then sends a `ChatMessage` to all participants announcing the newcomer.

That's it; we have a classic Mediator implementation. Before moving to the next section, let's take a look at the `Consumer` instance of `IChatRoom`, which is another integration test:

```
public class ChatRoomTest
{
    [Fact]
    public void ChatRoom_participants_should_send_and_receive_messages()
    {
        // Arrange
        var (kingChat, king) = CreateTestUser("King");
        var (kelleyChat, kelley) = CreateTestUser("Kelley");
        var (daveenChat, daveen) = CreateTestUser("Daveen");
        var (rutterChat, _) = CreateTestUser("Rutter");
        var chatroom = new ChatRoom();
```

We created four users with their respective `TestMessageWriter` instances in the `Arrange` section, as we did before (names were also randomly generated).

```
        // Act
        chatroom.Join(king);
        chatroom.Join(kelley);
        king.Send("Hey!");
        kelley.Send("What's up King?");
        chatroom.Join(daveen);
        king.Send("Everything is great, I joined the CrazyChatRoom!");
```

```
daveen.Send("Hey King!");
king.Send("Hey Daveen");
```

In the `Act` block, our test users join the `chatroom` instance and send messages into it.

```
// Assert
Assert.Empty(rutterChat.Output.ToString());
```

Since Rutter did not join the chatroom, we expect no message.

```
Assert.Equal(@"[King]: Has joined the channel
[Kelley]: Has joined the channel
[King]: Hey!
[Kelley]: What's up King?
[Daveen]: Has joined the channel
[King]: Everything is great, I joined the CrazyChatRoom!
[Daveen]: Hey King!
[King]: Hey Daveen
", kingChat.Output.ToString());
```

Since King is the first to join the channel, he is expected to have received all messages.

```
Assert.Equal(@"[Kelley]: Has joined the channel
[King]: Hey!
[Kelley]: What's up King?
[Daveen]: Has joined the channel
[King]: Everything is great, I joined the CrazyChatRoom!
[Daveen]: Hey King!
[King]: Hey Daveen
", kelleyChat.Output.ToString());
```

Kelley was the second user to join the chatroom, so the output contains almost all messages, except the line saying `[King]: Has joined the channel`.

```
Assert.Equal(@"[Daveen]: Has joined the channel
[King]: Everything is great, I joined the CrazyChatRoom!
[Daveen]: Hey King!
[King]: Hey Daveen
", daveenChat.Output.ToString());
}
```

Daveen joined after King and Kelley exchanged a few words, so the conversation is expected to start later.

```
private (TestMessageWriter, User) CreateTestUser(string name)
{
    var writer = new TestMessageWriter();
    var user = new User(writer, name);
    return (writer, user);
}
```

The `CreateTestUser` method helps simplify the `Arrange` section of the test case, similar to before.

```

private class TestMessageWriter : IMessageWriter<ChatMessage>
{
    public StringBuilder Output { get; } = new StringBuilder();
    public void Write(ChatMessage message)
    {
        Output.AppendLine($"[{message.Sender.Name}]: {message.Content}");
    }
}
} // Close the ChatRoomTest class
// As a reference, the IMessageWriter interface
// is the same as the previous project.
public interface IMessageWriter<TMessage>
{
    void Write(TMessage message);
}

```

The `TestMessageWriter` implementation is the same as the previous example, accumulating messages in a `StringBuilder` instance.

To summarize the test case, we had four users; three of them joined the same chatroom at a different time and chatted a little. The output is different for everyone since the time you join now matters. All participants are loosely coupled, thanks to the Mediator pattern, allowing us to extend the system without impacting the existing pieces. Leveraging the Mediator pattern can help us create more maintainable systems; many small pieces are easier to manage and to test than a large component handling all the logic itself.

Conclusion

As we explored in the two preceding projects, a **mediator** allows us to decouple the components of our system. **The mediator is the middleman between colleagues**, and it served us well in the small chatroom samples where each colleague can talk to the others without knowing how and without any need of even knowing them.

Now let's see how the Mediator pattern can help us follow the **SOLID** principles:

- **S:** The mediator extracts the communication responsibility from colleagues.
- **O:** With a mediator relaying the messages, we can create new colleagues and change the existing colleagues' behaviors without impacting the others. If we need a new colleague, we can register one with the mediator, and voilà! Moreover, if we need new mediation behavior, we can implement a new mediator and reuse the existing colleagues' implementations.
- **L:** N/A
- **I:** The system is divided into multiple small interfaces (`IMediator` and `IColleague`).
- **D:** All actors of the Mediator pattern solely depend on other interfaces.

Next, we explore CQRS, which allows us to separate commands and queries, leading to a more maintainable application. After all, all operations are queries or commands, no matter how we call them.

Implementing the CQRS pattern

CQRS stands for **Command Query Responsibility Segregation**. We can apply CQRS in two ways:

- Dividing requests into commands and queries.
- Applying the CQRS concept to a higher level, leading to a distributed system.

We stick with the first one here and tackle the second definition in *Chapter 16, Introduction to Microservices Architecture*.

Goal

The goal is to divide all requests into two categories: commands and queries.

- **A command mutates the state of an application.** For example, creating, updating, and deleting an entity are commands. In theory, commands do not return a value. In practice, they often do.
- **A query reads the state of the application but never changes it.** For example, reading an order, reading your order history, and retrieving your user profile are all queries.

Dividing operations into mutator requests (write/command) and accessor requests (read/query) creates a clear separation of concerns, leading us toward the SRP.

Design

There is no definite design for this, but for us, the flow of a command should look like the following:

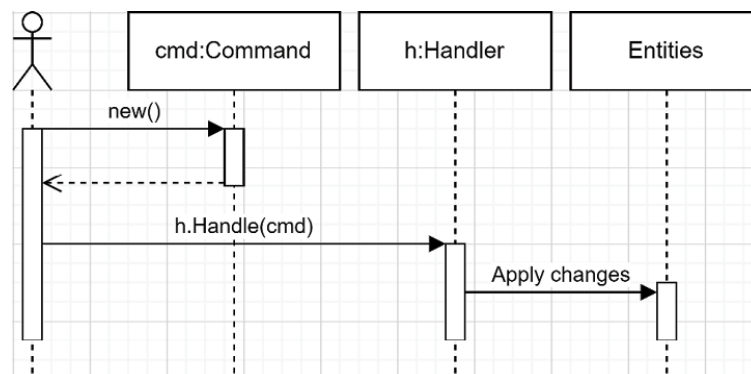


Figure 14.8: Sequence diagram representing the abstract flow of a command

The consumer creates a command object and sends it to a command handler, applying mutation to the application. In this case, I called it `Entities`, but it could have sent a SQL `UPDATE` command to a database or a web API call over HTTP; the implementation details do not matter.

The concept is the same for a query, but it returns a value instead. Very importantly, the query must not change the state of the application but query for data instead, like this:

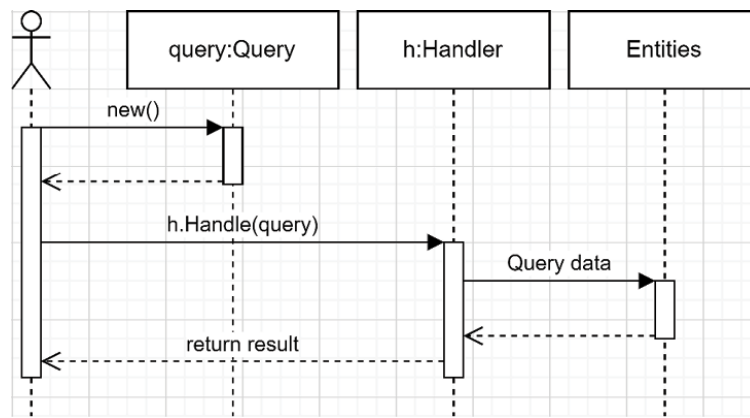


Figure 14.9: Sequence diagram representing the abstract flow of a query

Like the command, the consumer creates a query object and sends it to a handler, which then executes some logic to retrieve and return the requested data. You can replace `Entities` with anything that your handler needs to query the data.

Enough talk—let’s look at the CQRS project.

Project – CQRS

Context: We need to build an improved version of our chat system. The old system worked so well that we now need to scale it up. The mediator was of help to us, so we kept that part, and we picked CQRS to help us with this new, improved design. A participant was limited to a single chatroom in the past, but now a participant should be able to chat in multiple rooms simultaneously.

The new system is composed of three commands and two queries:

- A participant must be able to join a chatroom.
- A participant must be able to leave a chatroom.
- A participant must be able to send a message into a chatroom.
- A participant must be able to obtain the list of participants that joined a chatroom.
- A participant must be able to retrieve the existing messages from a chatroom.

The first three are commands, and the last two are queries. The system is backed by a mediator that makes heavy use of C# generics as follows:

```

public interface IMediator
{
    TReturn Send<TQuery, TReturn>(TQuery query)
        where TQuery : IQuery<TReturn>;
    void Send<TCommand>(TCommand command)
        where TCommand : ICommand;
    void Register<TCommand>(ICommandHandler<TCommand> commandHandler)
        where TCommand : ICommand;
    void Register<TQuery, TReturn>(IQueryHandler<TQuery, TReturn> commandHandler)
        where TQuery : IQuery<TReturn>;
}

public interface ICommand { }
public interface ICommandHandler<TCommand>
where TCommand : ICommand
{

```

```

        void Handle(TCommand command);
    }
    public interface IQuery<TReturn> { }
    public interface IQueryHandler<TQuery, TReturn>
    where TQuery : IQuery<TReturn>
    {
        TReturn Handle(TQuery query);
    }

```

If you are not familiar with generics, this might look daunting, but that code is way simpler than it looks. First, we have two empty interfaces: `ICommand` and `IQuery<TReturn>`. We could omit them, but they help identify the commands and the queries; they help describe our intent.

Then we have two interfaces that handle commands or queries. Let's start with the interface to implement for each type of command that we want to handle:

```

public interface ICommandHandler<TCommand>
where TCommand : ICommand
{
    void Handle(TCommand command);
}

```

That interface defines a `Handle` method that takes the command as a parameter. The generic parameter `TCommand` represents the type of command handled by the class implementing the interface. The query handler interface is the same, but it specifies a return value as well:

```

public interface IQueryHandler<TQuery, TReturn>
where TQuery : IQuery<TReturn>
{
    TReturn Handle(TQuery query);
}

```

The mediator abstraction allows registering command and query handlers using the generic interfaces that we just explored. It also supports sending commands and queries. Then we have the `ChatMessage` class, which is similar to the last two samples (with an added creation date):

```

public class ChatMessage
{
    public ChatMessage(IParticipant sender, string message)
    {
        Sender = sender ?? throw new ArgumentNullException(nameof(sender));
        Message = message ?? throw new ArgumentNullException(nameof(message));
        Date = DateTime.UtcNow;
    }
    public DateTime Date { get; }
    public IParticipant Sender { get; }
    public string Message { get; }
}

```

Next is the updated `IParticipant` interface:

```

public interface IParticipant
{
    string Name { get; }
}

```

```

void Join(IChatRoom chatRoom);
void Leave(IChatRoom chatRoom);
void SendMessageTo(IChatRoom chatRoom, string message);
void NewMessageReceivedFrom(IChatRoom chatRoom, ChatMessage message);
IEnumerable<IParticipant> ListParticipantsOf(IChatRoom chatRoom);
IEnumerable<ChatMessage> ListMessagesOf(IChatRoom chatRoom);
}

```

All methods of the `IParticipant` interface accept an `IChatRoom` parameter to support multiple chatrooms. Then, the updated `IChatRoom` interface has a name and a few basic operations to meet the requirement of a chatroom, like adding and removing participants:

```

public interface IChatRoom
{
    string Name { get; }
    void Add(IParticipant participant);
    void Remove(IParticipant participant);
    IEnumerable<IParticipant> ListParticipants();
    void Add(ChatMessage message);
    IEnumerable<ChatMessage> ListMessages();
}

```

Before going into commands and the chat itself, let's take a peek at the Mediator class:

```

public class Mediator : IMediator
{
    private readonly HandlerDictionary _handlers = new
        HandlerDictionary();
    public void Register<TCommand>(ICommandHandler<TCommand> commandHandler)
        where TCommand : ICommand
    {
        _handlers.AddHandler(commandHandler);
    }
    public void Register<TQuery, TReturn>(IQueryHandler<TQuery, TReturn> commandHandler)
        where TQuery : IQuery<TReturn>
    {
        _handlers.AddHandler(commandHandler);
    }
    public TReturn Send<TQuery, TReturn>(TQuery query)
        where TQuery : IQuery<TReturn>
    {
        var handler = _handlers.Find<TQuery, TReturn>();
        return handler.Handle(query);
    }
    public void Send<TCommand>(TCommand command)
        where TCommand : ICommand
    {
        var handlers = _handlers.FindAll<TCommand>();
        foreach (var handler in handlers)
        {
            handler.Handle(command);
        }
    }
}

```

The `Mediator` class supports registering commands and queries as well as sending a query to a handler or sending a command to zero or more handlers.

Note

I omitted the implementation of `HandlerDictionary` because it does not add anything, and it is just an implementation detail. It is available on GitHub (<https://adpg.Link/CWCe>).

Now to the commands. I grouped the commands and the handlers together to keep it organized and readable, but you could use another way to organize yours. Moreover, since this is a small project, all the commands are in the same file, which would not be viable for something bigger. Remember we are playing LEGO® blocks, this chapter covers the CQRS pieces, but you can always use them with bigger pieces like Clean Architecture or other types of architecture. Let's start with the `JoinChatRoom` class:

```
public class JoinChatRoom
{
    public class Command : ICommand
    {
        public Command(IChatRoom chatRoom, IParticipant requester)
        {
            ChatRoom = chatRoom ?? throw new ArgumentNullException(nameof(chatRoom));
            Requester = requester ?? throw new ArgumentNullException(nameof(requester));
        }
        public IChatRoom ChatRoom { get; }
        public IParticipant Requester { get; }
    }
    public class Handler : ICommandHandler<Command>
    {
        public void Handle(Command command)
        {
            command.ChatRoom.Add(command.Requester);
        }
    }
}
```

The `JoinChatRoom.Command` class represents the command itself, a data structure that carries the command data. The

`JoinChatRoom.Handler` class handles that type of command. When executed, it adds the specified `IParticipant` to the specified `IChatRoom`, from the `ChatRoom` and `Requester` properties (highlighted line). Next command:

```
public class LeaveChatRoom
{
    public class Command : ICommand
    {
        public Command(IChatRoom chatRoom, IParticipant requester)
        {
            ChatRoom = chatRoom ?? throw new ArgumentNullException(nameof(chatRoom));
            Requester = requester ?? throw new
ArgumentNullException(nameof(requester));
        }
        public IChatRoom ChatRoom { get; }
        public IParticipant Requester { get; }
    }
    public class Handler : ICommandHandler<Command>
    {
        public void Handle(Command command)
```



```

        {
            command.ChatRoom.Remove(command.Requester);
        }
    }
}

```

That code represents the exact opposite of the `JoinChatRoom` command, the `LeaveChatRoom` handler removes an `IParticipant` from the specified `IChatRoom` (highlighted line). To the next command:

```

public class SendChatMessage
{
    public class Command : ICommand
    {
        public Command(IChatRoom chatRoom, ChatMessage message)
        {
            ChatRoom = chatRoom ?? throw new ArgumentNullException(nameof(chatRoom));
            Message = message ?? throw new ArgumentNullException(nameof(message));
        }
        public IChatRoom ChatRoom { get; }
        public ChatMessage Message { get; }
    }
    public class Handler : ICommandHandler<Command>
    {
        public void Handle(Command command)
        {
            command.ChatRoom.Add(command.Message);
            foreach (var participant in command.ChatRoom.ListParticipants())
            {
                participant.NewMessageReceivedFrom(
                    command.ChatRoom,
                    command.Message
                );
            }
        }
    }
}

```

The `SendChatMessage` command, on the other hand, handles two things (highlighted lines):

- It adds the specified `Message` to `IChatRoom` (which is now only a data structure that keeps track of users and past messages).
- It also sends the specified `Message` to all `IParticipant` instances that joined that `IChatRoom`.

We are starting to see many smaller pieces interacting with each other to create a more developed system. But we are not done; let's look at the two queries, then the chat implementation:

```

public class ListParticipants
{
    public class Query : IQuery<IEnumerable<IParticipant>>
    {
        public Query(IChatRoom chatRoom, IParticipant requester)
        {
            Requester = requester ?? throw new ArgumentNullException(nameof(requester));
            ChatRoom = chatRoom ?? throw new ArgumentNullException(nameof(chatRoom));
        }
        public IParticipant Requester { get; }
        public IChatRoom ChatRoom { get; }
    }
}

```

```

    }
    public class Handler : IQueryHandler<Query, IEnumerable<IParticipant>>
    {
        public IEnumerable<IParticipant> Handle(Query query)
        {
            return query.ChatRoom.ListParticipants();
        }
    }
}

```

The `ListParticipants` query's handler uses the specified `IChatRoom` and returns its participants (highlighted line). Now, to the last query:

```

public class ListMessages
{
    public class Query : IQuery<IEnumerable<ChatMessage>>
    {
        public Query(IChatRoom chatRoom, IParticipant requester)
        {
            Requester = requester ?? throw new ArgumentNullException(nameof(requester));
            ChatRoom = chatRoom ?? throw new ArgumentNullException(nameof(chatRoom));
        }
        public IParticipant Requester { get; }
        public IChatRoom ChatRoom { get; }
    }
    public class Handler : IQueryHandler<Query, IEnumerable<ChatMessage>>
    {
        public IEnumerable<ChatMessage> Handle(Query query)
        {
            return query.ChatRoom.ListMessages();
        }
    }
}

```

The `ListMessages` query's handler uses the specified `IChatRoom` instance and returns its messages.

Note

All of the commands and queries reference `IParticipant` so we could enforce rules such as “`IParticipant` must join a channel before sending messages,” for example. I decided to omit these details to keep the code simple, but feel free to add those features if you want to.

Next, let's take a look at the `ChatRoom` class, which is a simple data structure that holds the state of a chatroom:

```

public class ChatRoom : IChatRoom
{
    private readonly List<IParticipant> _participants = new List<IParticipant>();
    private readonly List<ChatMessage> _chatMessages = new List<ChatMessage>();
    public ChatRoom(string name)
    {
        Name = name ?? throw new ArgumentNullException(nameof(name));
    }
    public string Name { get; }
    public void Add(IParticipant participant)
    {
        _participants.Add(participant);
    }
}

```

```

    }
    public void Add(ChatMessage message)
    {
        _chatMessages.Add(message);
    }
    public IEnumerable<ChatMessage> ListMessages()
    {
        return _chatMessages.AsReadOnly();
    }
    public IEnumerable<IParticipant> ListParticipants()
    {
        return _participants.AsReadOnly();
    }
    public void Remove(IParticipant participant)
    {
        _participants.Remove(participant);
    }
}

```

If we take a second look at the `ChatRoom` class, it has a `Name` property, and it contains a list of `IParticipant` instances and a list of `ChatMessage` instances. Both `ListMessages()` and `ListParticipants()` return the list `AsReadOnly()` so a clever programmer cannot mutate the state of `ChatRoom` from the outside. That's it, the new `ChatRoom` class is a façade over its underlying dependencies.

Finally, the `Participant` class is probably the most exciting part of this system because it is the one that makes heavy use of our **Mediator** and **CQRS** implementations:

```

public class Participant : IParticipant
{
    private readonly IMediator _mediator;
    private readonly IMessageWriter _messageWriter;
    public Participant(IMediator mediator, string name, IMessageWriter messageWriter)
    {
        _mediator = mediator ?? throw new ArgumentNullException(nameof(mediator));
        Name = name ?? throw new ArgumentNullException(nameof(name));
        _messageWriter = messageWriter ?? throw new ArgumentNullException(nameof(messageWriter));
    }
    public string Name { get; }
    public void Join(IChatRoom chatRoom)
    {
        _mediator.Send(new JoinChatRoom.Command(chatRoom, this));
    }
    public void Leave(IChatRoom chatRoom)
    {
        _mediator.Send(new LeaveChatRoom.Command(chatRoom, this));
    }
    public IEnumerable<ChatMessage> ListMessagesOf(IChatRoom chatRoom)
    {
        return _mediator.Send<ListMessages.Query, IEnumerable<ChatMessage>>(new ListMessages.Query(chatRoom));
    }
    public IEnumerable<IParticipant> ListParticipantsOf(IChatRoom chatRoom)
    {
        return _mediator.Send<ListParticipants.Query, IEnumerable<IParticipant>>(new ListParticipants.Query(chatRoom));
    }
    public void NewMessageReceivedFrom(IChatRoom chatRoom, ChatMessage message)
    {
        _messageWriter.Write(chatRoom, message);
    }
    public void SendMessageTo(IChatRoom chatRoom, string message)
    {
    }
}

```

```

        _mediator.Send(new SendChatMessage.Command(chatRoom, new ChatMessage(this, me
    }
}

```

Every method of the `Participant` class, apart from `NewMessageReceivedFrom`, sends a command or a query through `IMediator`, breaking the tight coupling between `Participant` and the system's operations (that is, the commands and queries). If we think about it, the `Participant` class is also a simple façade over its underlying dependencies, delegating most of the work to the mediator.

Now, let's look at how it works when everything is put together. I grouped several test cases that share the following setup code:

```

public class ChatRoomTest
{
    private readonly IMediator _mediator = new Mediator();
    private readonly TestMessageWriter _reagenMessageWriter = new();
    private readonly TestMessageWriter _garnerMessageWriter = new();
    private readonly TestMessageWriter _corneliaMessageWriter = new();
    private readonly IChatRoom _room1 = new ChatRoom("Room 1");
    private readonly IChatRoom _room2 = new ChatRoom("Room 2");
    private readonly IParticipant _reagen;
    private readonly IParticipant _garner;
    private readonly IParticipant _cornelia;
    public ChatRoomTest()
    {
        _mediator.Register(new JoinChatRoom.Handler());
        _mediator.Register(new LeaveChatRoom.Handler());
        _mediator.Register(new SendChatMessage.Handler());
        _mediator.Register(new ListParticipants.Handler());
        _mediator.Register(new ListMessages.Handler());
        _reagen = new Participant(_mediator, "Reagen", _reagenMessageWriter);
        _garner = new Participant(_mediator, "Garner", _garnerMessageWriter);
        _cornelia = new Participant(_mediator, "Cornelia", _corneliaMessageWriter);
    }
    // Omitted test cases and helpers
}

```

The test program setup is composed of the following:

- One `IMediator`, which enables all colleagues to interact with each other.
- Two `IChatRoom` instances.
- Three `IParticipant` instances and their `TestMessageWriter`.

In the constructor, all handlers are registered with the `Mediator` instance, so it knows how to handle commands and queries. The names of the participants are randomly generated. The `TestMessageWriter` implementation accumulates the data in a list of tuples (`List<(IChatRoom, ChatMessage)>`) to assess what is sent to what participant:

```

private class TestMessageWriter : IMessageWriter
{
    public List<(IChatRoom chatRoom, ChatMessage message)> Output { get; } = new();
    public void Write(IChatRoom chatRoom, ChatMessage message)
    {
        Output.Add((chatRoom, message));
    }
}

```

```

    }
}

```

Here is the first test case:

```

[Fact]
public void A_participant_should_be_able_to_list_the_participants_that_joined_a_chatroom()
{
    _reagen.Join(_room1);
    _reagen.Join(_room2);
    _garner.Join(_room1);
    _cornelia.Join(_room2);
    var room1Participants = _reagen.ListParticipantsOf(_room1);
    Assert.Collection(room1Participants,
        p => Assert.Same(_reagen, p),
        p => Assert.Same(_garner, p)
    );
}

```

In that test case, Reagen and Garner join Room 1, and Reagen and Cornelia join Room 2. Then Reagen requests the list of participants from Room 1, which outputs Reagen and Garner. The code is easy to understand and use. Under the hood, it uses commands and queries through a mediator, breaking tight coupling between the colleagues. Here is a sequence diagram representing what is happening when a participant joins a chatroom:

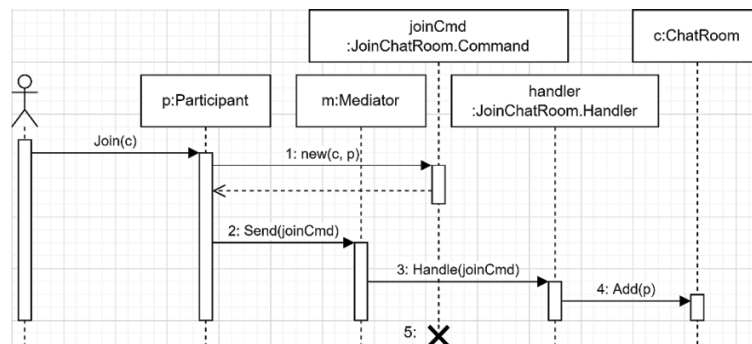


Figure 14.10: Sequence diagram representing the flow of a participant (*p*) joining a chatroom (*c*)

1. The participant (*p*) creates a `JoinChatRoom` command (`joinCmd`).
2. *p* sends `joinCmd` through the mediator (*m*).
3. *m* finds and dispatches `joinCmd` to its handler (`handler`).
4. `handler` executes the logic (adds *p* to the chatroom).
5. `joinCmd` ceases to exist afterward; commands are ephemeral.

That means `Participant` never interacts directly with `ChatRoom` or other participants.

Then a similar workflow happens when a participant requests the list of participants of a chatroom:

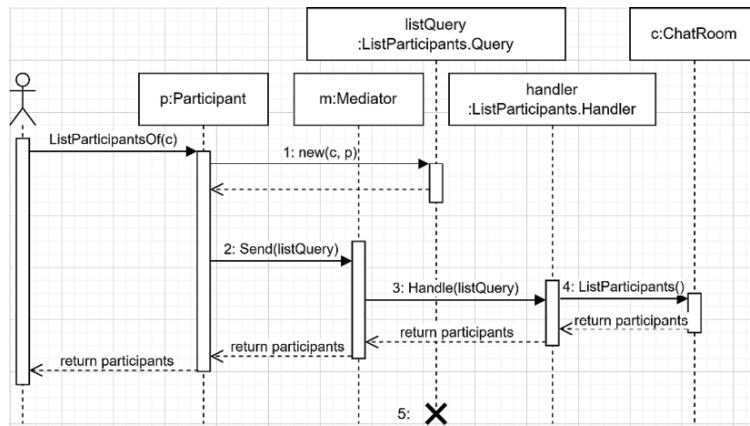


Figure 14.11: Sequence diagram representing the flow of a participant (p) requesting the list of participants of a chatroom (c)

1. Participant (p) creates a ListParticipants query (listQuery).
2. p sends listQuery through the mediator (m).
3. m finds and dispatches the query to its handler (handler).
4. handler executes the logic (lists the participants of the chatroom).
5. listQuery ceases to exist afterward; queries are also ephemeral.

Once again, Participant does not interact directly with ChatRoom.

Here is another test case where Participant sends a message to a chatroom, and another Participant receives it:

```

[Fact]
public void A_participant_should_receive_new_messages()
{
    _reagen.Join(_room1);
    _garner.Join(_room1);
    _garner.Join(_room2);
    _reagen.SendMessageTo(_room1, "Hello!");
    Assert.Collection(_garnerMessageWriter.Output,
        line =>
        {
            Assert.Equal(_room1, line.chatRoom);
            Assert.Equal(_reagen, line.message.Sender);
            Assert.Equal("Hello!", line.message.Message);
        }
    );
}

```

In that test case, Reagen joins Room 1 while Garner joins Rooms 1 and 2. Then Reagen sends a message to Room 1, and we verify that Garner received it once. The `SendMessageTo` workflow is very similar to the other one that we saw, but with a more complex command handler:

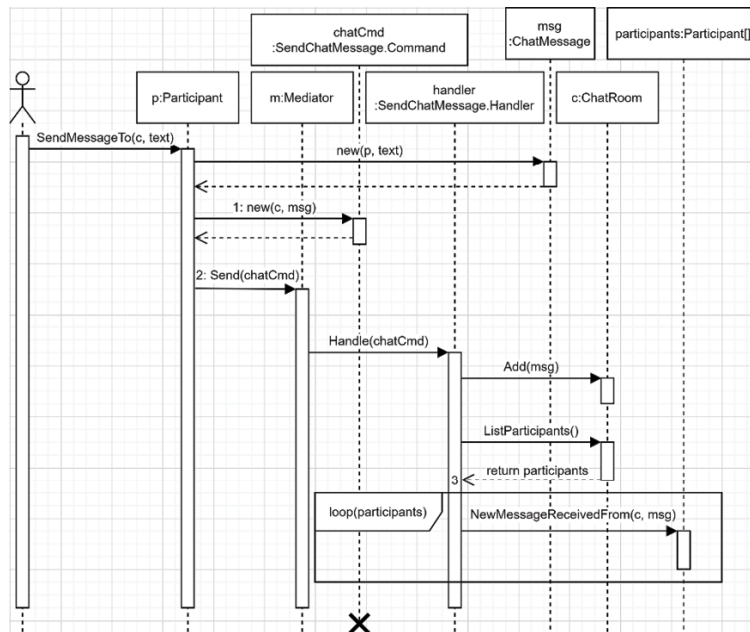


Figure 14.12: Sequence diagram representing the flow of a participant (*p*) sending a message (*msg*) to a chatroom (*c*)

From that diagram, we can observe that the logic was pushed to the `ChatMessage.Handler` class. All of the other actors work together with limited knowledge of each other (or even no knowledge of each other).

This demonstrates how CQRS works with a mediator:

1. A consumer (the participant in this case) creates a command (or a query).
2. The consumer sends that command through the mediator.
3. The mediator sends that command to one or more handlers, each executing their piece of logic for that command.

You can explore the other test cases to familiarize yourself with the program and the concepts.

Note

You can debug the tests in Visual Studio; use breakpoints combined with *Step Into (F11)* and *Step Over (F10)* to explore the sample.

I also created a `ChatModerator` instance that sends a message in a “moderator chatroom” when a message contains a word from the `bad-words` collection. That test case executes multiple handlers for each `SendChatMessage.Command`. I’ll leave you to explore these other test cases yourself, so we don’t wander astray from our goal.

Before concluding CQRS, let’s peek at marker interfaces.

Code smell – Marker Interfaces

We used the empty `ICommand` and `IQuery<TReturn>` interfaces in the code samples to make the code more explicit and self-descriptive. Empty interfaces are a sign that something may be wrong: a code smell. We call those **marker interfaces**.

In our case, they help identify commands and queries but are empty and add nothing. We could discard them without any impact on our system. On the other hand, we are not performing any kind of magic tricks or violating any principles, so it is OK to have them; they help define the intent. Moreover, we could leverage them to make the code more dynamic, like leveraging dependency injection to register handlers. Furthermore, I designed those interfaces this way as a bridge to the next project you are about to see.

Back to the marker interfaces, here are two types of marker interfaces that are code smells in C#:

- Metadata
- Dependency identifier

Metadata

Markers can be used to define metadata. A class “implements” the empty interface, and some consumer does something with it later. It could be an assembly scanning for specific types, a choice of strategy, or something else.

Instead of creating marker interfaces to add metadata, try to use custom attributes. The idea behind attributes is to add metadata to classes or members. On the other hand, interfaces exist to create a contract, and they should define at least one member; empty contracts are like a blank sheet.

In a real-world scenario, you may want to consider the cost of one versus the other. Markers are very cheap to implement but can violate the SOLID principles. Attributes could be as cheap to implement if the mechanism is already implemented or supported by the framework but could cost a lot more than a marker interface to implement if you need to program everything by hand. Before deciding, you must evaluate money, time, and the skills required as crucial factors.

Dependency identifier

If you need markers to inject some specific dependency in a particular class, you are most likely cheating the **inversion of control**. Instead, you should find a way to achieve the same goal using dependency injection, such as by contextually injecting your dependencies.

Let's start with the following interface:

```
public interface IStrategy
{
    string Execute();
}
```

In our program, we have two implementations and two markers, one for each implementation:

```
public interface IStrategyA : IStrategy { }
public interface IStrategyB : IStrategy { }
public class StrategyA : IStrategyA
{
    public string Execute() => "StrategyA";
}
```



```

    }
    public class StrategyB : IStrategyB
    {
        public string Execute() => "StrategyB";
    }

```

The code is barebones, but all the building blocks are there:

- StrategyA implements IStrategyA, which inherits from IStrategy.
- StrategyB implements IStrategyB, which inherits from IStrategy.
- Both IStrategyA and IStrategyB are empty marker interfaces.

Now, the consumer needs to use both strategies, so instead of controlling dependencies from the composition root, the consumer requests the markers:

```

public class Consumer
{
    public IStrategyA StrategyA { get; }
    public IStrategyB StrategyB { get; }
    public Consumer(IStrategyA strategyA, IStrategyB strategyB)
    {
        StrategyA = strategyA ?? throw new ArgumentNullException(nameof(strategyA));
        StrategyB = strategyB ?? throw new ArgumentNullException(nameof(strategyB));
    }
}

```

The `Consumer` class exposes the strategies through properties to assert its composition later. Let's test that out by building a dependency tree, simulating the composition root, and then asserting the value of the consumer properties:

```

[Fact]
public void ConsumerTest()
{
    // Arrange
    var serviceProvider = new ServiceCollection()
        .AddSingleton<IStrategyA, StrategyA>()
        .AddSingleton<IStrategyB, StrategyB>()
        .AddSingleton<Consumer>()
        .BuildServiceProvider();

    // Act
    var consumer = serviceProvider.GetRequiredService<Consumer>();

    // Assert
    Assert.IsType<StrategyA>(consumer.StrategyA);
    Assert.IsType<StrategyB>(consumer.StrategyB);
}

```

Both properties are of the expected type, but that is not the problem. The `Consumer` class controls what dependencies to use and when to use them by injecting markers A and B instead of two `IStrategy` instances. Due to that, we cannot control the dependency tree from the composition root. For example, we cannot change `IStrategyA` to `IStrategyB` and `IStrategyB` to `IStrategyA`, nor inject two `IStrategyB` instances or two `IStrategyA` instances, nor even create an `IStrategyC` instance to replace `IStrategyA` or `IStrategyB`.

How do we fix this? Let's start by deleting our markers and injecting two `IStrategy` instances instead (the changes are highlighted). After doing that, we end up with the following object structure:

```
public class StrategyA : IStrategy
{
    public string Execute() => "StrategyA";
}
public class StrategyB : IStrategy
{
    public string Execute() => "StrategyB";
}
public class Consumer
{
    public IStrategy StrategyA { get; }
    public IStrategy StrategyB { get; }
    public Consumer(IStrategy strategyA, IStrategy strategyB)
    {
        StrategyA = strategyA ?? throw new ArgumentNullException(nameof(strategyA));
        StrategyB = strategyB ?? throw new ArgumentNullException(nameof(strategyB));
    }
}
```

The `Consumer` class no longer controls the narrative with the new implementation, and the composition responsibility falls back to the composition root. Unfortunately, there is no way to do contextual injections using the default dependency injection container, and I don't want to get into a third-party framework for this. But all is not lost yet; we can use a factory to help ASP.NET Core build the `Consumer` instance, like this:

```
// Arrange
var serviceProvider = new ServiceCollection()
    .AddSingleton<StrategyA>()
    .AddSingleton<StrategyB>()
    .AddSingleton(serviceProvider =>
    {
        var strategyA = serviceProvider.GetRequiredService<StrategyA>();
        var strategyB = serviceProvider.GetRequiredService<StrategyB>();
        return new Consumer(strategyA, strategyB);
    })
    .BuildServiceProvider();
// Act
var consumer = serviceProvider.GetRequiredService<Consumer>();
// Assert
Assert.IsType<StrategyA>(consumer.StrategyA);
Assert.IsType<StrategyB>(consumer.StrategyB);
```

From that point forward, we control the program's composition, and we can swap A with B or do anything else that we want to, as long as the implementation respects the `IStrategy` contract.

To conclude, using markers instead of doing contextual injection breaks the inversion of control principle, making the consumer control its dependencies. That's very close to using the `new` keyword to instantiate objects. Inverting the dependency control back is easy, even using the default container.

If you need to inject dependencies contextually, I started an open source project in 2020 that does that. Multiple other third-party libraries add features or replace the default IoC container altogether if needed. See the *Further reading* section.

Conclusion

CQRS suggests dividing the operations of a program into **commands** and **queries**. A command mutates data, and a query returns data. We can apply the **Mediator** pattern to break the tight coupling between the pieces of the CQRS program, like sending the commands and queries.

Dividing the program helps separate the different pieces and focus on the commands and queries that travel from a consumer through the mediator to one or more handlers. The data contract of commands and queries becomes the program's backbone, trimming down the coupling between objects and tying them to those thin data structures instead, leaving the central piece (the mediator) to manage the links between them.

On the other hand, you may find the codebase more intimidating when using CQRS due to the multiple classes. However, keep in mind that each of those classes does less (having a single responsibility), making them easier to test than a more sizable class with many responsibilities. The way you organize the classes should also greatly help. We organize the pieces a certain way in the book, but you could organize them differently in a different context.

Now let's see how CQRS can help us follow the **SOLID** principles:

- **S**: Dividing an application into commands, queries, and handlers takes us toward encapsulating single responsibilities into different classes.
- **O**: CQRS helps extend the software without modifying the existing code, such as adding handlers and creating new commands.
- **L**: N/A
- **I**: CQRS makes it easier to create multiple small interfaces with a clear distinction between commands, queries, and their respective handlers.
- **D**: N/A

Now that we have explored CQRS and the Mediator pattern, it is time to get lazy and look at a tool that will save us some hassle.

Using MediatR as a mediator

In this section, we are exploring MediatR, an open source mediator implementation.

What is MediatR? Let's start with its maker's description from its GitHub repository, which brands it as this:

“Simple, unambitious mediator implementation in .NET”

MediatR is a simple but very powerful tool doing in-process communication through messaging. It supports a request/response flow through com-

mands, queries, notifications, and events, synchronously and asynchronously.

You can install the NuGet package using the .NET CLI: `dotnet add package MediatR`.

Now that I have quickly introduced the tool, we are going to explore the migration of our Clean Architecture sample but instead use MediatR to dispatch the `StocksController` requests to the core use cases.

Why migrate our Clean Architecture sample? The primary reason we are building the same project using different models is for ease of comparison. It is much easier to compare the changes of the same features than if we were building completely different projects.

What are the advantages of using MediatR in this case? It allows us to organize the code around use cases (vertically) instead of services (horizontally), leading to more cohesive features. We remove the service layer (the `StockService` class) and replace it with multiple use cases instead (the `AddStocks` and `RemoveStock` classes). MediatR also enables an MVC-like pipeline, which we can extend by programming pipeline behaviors. Those extensibility points allow us to manage cross-cutting concerns, such as requests validation, centrally without impacting the consumers and use cases. We explore request validation in *Chapter 15, Getting Started with Vertical Slice Architecture*.

Let's jump into the code now to see how it works.

Project – Clean Architecture with MediatR

Context: We want to break some more of the coupling in the Clean Architecture project that we built in *Chapter 12, Understanding Layering*, by leveraging the **Mediator** pattern and a **CQRS**-inspired approach.

The clean architecture solution was already solid, but MediatR will pave the way to more good things later. The only “major” change is the replacement of the `StockService` with two feature objects, `AddStocks` and `RemoveStocks`, that we explore soon.

First, we must install the

`MediatR.Extensions.Microsoft.DependencyInjection` NuGet package in the web project. That package adds a helper method to scan one or more assemblies for MediatR handlers, preprocessors, and post-processors. It adds those to the IoC container with a transient lifetime.

With that package in hand, in the `Program.cs` file, we can do this:

```
builder.Services.AddMediatR(typeof(NotEnoughStockException).Assembly);
```

Note that the `NotEnoughStockException` class is part of the core project. We can also specify more than one assembly here; as of version 9.0.0 of MediatR, there are six overloads to that method. Moreover, I picked the `NotEnoughStockException` class, but I could have chosen any class from the `Core` assembly.

MediatR exposes two types of messages, request/response and notifications. The first model executes a single handler, while the second allows

multiple handlers to handle each message. The request/response model is perfect for both commands and queries, while notifications are more suited to an event-based model applying the Publish-Subscribe pattern. We cover the Publish-Subscribe pattern in *Chapter 16, Introduction to Microservices Architecture*.

Now that everything is “magically” registered, we can look at the use cases that replace the `StockService`. Let’s have a look at the updated `AddStocks` code first:

```
namespace Core.UseCases;
public class AddStocks
{
    public class Command : IRequest<int>
    {
        public int ProductId { get; set; }
        public int Amount { get; set; }
    }
    public class Handler : IRequestHandler<Command, int>
    {
        private readonly IProductRepository _productRepository;
        public Handler(IProductRepository productRepository)
        {
            _productRepository = productRepository ?? throw new ArgumentNullException()
        }
        public async Task<int> Handle(Command request, CancellationToken cancellation)
        {
            var product = await _productRepository.FindByIdAsync(request.ProductId, cancellation);
            if (product == null)
            {
                throw new ProductNotFoundException(request.ProductId);
            }
            product.AddStock(request.Amount);
            await _productRepository.UpdateAsync(product, cancellation);
            return product.QuantityInStock;
        }
    }
}
```

Since we covered both use cases in the previous chapters and the changes are very similar, we will analyze both together, after the `RemoveStocks` use case:

```
namespace Core.UseCases;
public class RemoveStocks
{
    public class Command : IRequest<int>
    {
        public int ProductId { get; set; }
        public int Amount { get; set; }
    }
    public class Handler : IRequestHandler<Command, int>
    {
        private readonly IProductRepository _productRepository;
        public Handler(IProductRepository productRepository)
        {
            _productRepository = productRepository ?? throw new ArgumentNullException()
        }
        public async Task<int> Handle(Command request, CancellationToken cancellation)
        {
            var product = await _productRepository.FindByIdAsync(request.ProductId, cancellation);
            if (product == null)
            {
                throw new ProductNotFoundException(request.ProductId);
            }
            product.RemoveStock(request.Amount);
            await _productRepository.UpdateAsync(product, cancellation);
            return product.QuantityInStock;
        }
    }
}
```

```

        {
            throw new ProductNotFoundException(request.ProductId);
        }
        product.RemoveStock(request.Amount);
        await _productRepository.UpdateAsync(product, cancellationToken);
        return product.QuantityInStock;
    }
}

```

As you may have noticed in the code, I chose the same pattern to build the commands as I did with the CQRS sample, so we have a class per use case containing two nested classes: `Command` and `Handler`. I find this structure makes for very clean code when you have a 1-on-1 relationship between the command class and its handler.

By using the MediatR request/response model, the command (or query) becomes a request and must implement the `IRequest<TResponse>` interface. The handlers must implement the `IRequestHandler<TRequest, TResponse>` interface. Instead, we can also implement the `IRequest` and `IRequestHandler<TRequest>` interfaces for a command that returns nothing (`void`).

Note

There are more options that are part of MediatR, and the documentation is complete enough for you to dig deeper by yourself. Not that I want to, but I must limit the subjects that I talk about or risk writing an encyclopedia.

Let's analyze the anatomy of the `AddStocks` use case. Here is the old code as a reference:

```

namespace Core.Services;
public class StockService
{
    private readonly IProductRepository _repository;
    // Omitted constructor
    public async Task<int> AddStockAsync(int productId, int amount, CancellationToken
    {
        var product = await _repository.FindByIdAsync(productId, cancellationToken);
        if (product == null)
        {
            throw new ProductNotFoundException(productId);
        }
        product.AddStock(amount);
        await _repository.UpdateAsync(product, cancellationToken);
        return product.QuantityInStock;
    }
    // Omitted RemoveStockAsync method
}

```

The first difference is that we moved the loose parameters (highlighted) into the `Command` class, which encapsulates the whole request:

```

public class Command : IRequest<int>
{
    public int ProductId { get; set; }
}

```

```

    public int Amount { get; set; }
}

```

Then the `Command` class specifies the handler's expected return value by implementing the `IRequest<TResponse>` interface, where `TResponse` is an `int`. That gives us a typed response when sending the request through MediatR. This is not "pure CQRS" because the command handler returns an integer representing the updated `QuantityInStock`. However, we could call that optimization since executing one command and one query would be overkill for this scenario (possibly leading to two database calls instead of one). Moreover, we are exploring MediatR using a CQRS-like approach, which is more than fine for in-process communication.

I'll skip the `RemoveStocks` use case to avoid repeating myself as it follows the same pattern. Instead, let's look at the consumption of those use cases. I omitted the exception handling to keep the code streamlined and because `try/catch` blocks would only add noise to the code in this case and hinder our study of the pattern:

```

app.MapPost("/products/{productId:int}/add-stocks", async (
    int productId,
    AddStocks.Command command,
    IMediator mediator,
    CancellationToken cancellationToken) =>
{
    command.ProductId = productId;
    var quantityInStock = await mediator.Send(command, cancellationToken);
    var stockLevel = new StockLevel(quantityInStock);
    return Results.Ok(stockLevel);
});
app.MapPost("/products/{productId:int}/remove-stocks", async (
    int productId,
    RemoveStocks.Command command,
    IMediator mediator,
    CancellationToken cancellationToken) =>
{
    command.ProductId = productId;
    var quantityInStock = await mediator.Send(command, cancellationToken);
    var stockLevel = new StockLevel(quantityInStock);
    return Results.Ok(stockLevel);
});
// Omitted code
public record class StockLevel(int QuantityInStock);

```

In both delegates, we inject an `IMediator` and a command object (highlighted). We also let ASP.NET Core inject a `CancellationToken`, which we pass to MediatR. The model binder loads the data from the HTTP request into the objects that we send using the `Send` method of the `IMediator` interface (highlighted). Then we map the result into the `StockLevel` DTO before returning its value and an HTTP status code of `200 OK`. The `StockLevel` record class is the same as before.

Note

The default model binder cannot load data from multiple sources. Because of that, we must inject `productId` and assign its value to the `command.ProductId` property manually. Even if both values could be taken from the body, the re-

source identifier of that endpoint would become less exhaustive (no `productId` in the URI).

With MVC, we could create a custom model binder.

With minimal APIs, we could create a static `BindAsync` method to manually do the model binding, which is not very extensible and would tightly couple the `Core` assembly with the `HttpContext`. I suppose we will need to wait for .NET 7+ to get improvements into that field.

Meanwhile, Damian Edwards, Principal Program Manager Architect for the .NET platform, has an experimental project underway that allows us to create binders like this:

```
public class AddStocksCommandBinder : IParameterBinder<AddStocks.Command>
{
    public async ValueTask<AddStocks.Command?> BindAsync(HttpContext context)
    {
        if (!context.Request.HasJsonContentType())
        {
            throw new BadRequestException(
                "The content-type must be JSON.",
                StatusCodes.Status415UnsupportedMediaType
            );
        }
        var command = await context.Request
            .ReadFromJsonAsync<AddStocks.Command>(context.RequestAborted);
        if (command is null)
        {
            throw new BadRequestException(
                "The request body must contain an AddStocks Command."
            );
        }
        command.ProductId = int.Parse(context.GetRouteValue("productId")?.ToString() ?? "");
        return command;
    }
}
```

Then we need to register it with the container:

```
builder.Services
    .AddParameterBinder<AddStocksCommandBinder, AddStocks.Command>()
```

Such an approach allows breaking tight coupling between the class we need custom binding for and the code that contains the model binding logic. For example, we could have multiple components that reuse the same input model but leverage a different model binder to translate the data, like XML, JSON, or gRPC, to name a few. I've left the link in the *further reading* section at the end of the chapter.

All in all, it is almost the same code as before, but we use MediatR, the Mediator pattern, and a CQRS-inspired style instead.

Conclusion

With MediatR, we packed the power of a CQRS-inspired pipeline with the Mediator pattern into a Clean Architecture application. We were able to break the coupling between the request delegates and the use case handler (previously a service). A simple DTO such as a command object makes delegates (or controllers in the case of MVC) unaware of the handlers, leaving MediatR to be the middleman between the commands and their handlers. Due to that, the handlers could change along the way without having any impact on the controller.

Moreover, we could configure more interaction between the command and the handler with `IRequestPreProcessor`, `IRequestPostProcessor`, and `IRequestExceptionHandler`. These allow us to extend the MediatR request pipeline with crosscutting concerns like validation.

MediatR helps us follow the SOLID principles the same way as the Mediator and CQRS patterns combined. The only drawback (if it is one) is that the use cases now also control the API's input contracts. The command objects are now the input DTOs. If this is something that you must avoid in a project, you can input a DTO in the action method instead, create the command object, then send it to MediatR.

Summary

In this chapter, we looked at the Mediator pattern. That pattern allows us to cut the ties between collaborators, mediating the communication between them. Then we attacked the CQRS pattern, which advises the division of software behaviors into commands and queries. Those two patterns are tools that cut tight coupling between components.

Then we updated a Clean Architecture project to use MediatR, an open source generic Mediator implementation that is CQRS-oriented. There are many more possible uses than what we explored, but this is still a great start. This concludes another chapter where we explored techniques to break tight coupling and divide systems into smaller parts.

All of those building blocks are leading us to the next chapter, where we will be piecing all of those patterns and tools together to explore the Vertical Slice Architecture.

Questions

Let's take a look at a few practice questions:

1. Can we use a mediator inside a colleague to call another colleague?
2. In CQRS, can a command return a value?
3. How much does MediatR cost?
4. Imagine a design has a marker interface to add metadata to some classes. Do you think you should review that design?

Further reading

Here are a few links to build on what we have learned in the chapter:

- MediatR: <https://adpg.link/ZQap>
- To get rid of setting `ProductId` manually in the Clean Architecture with MediatR project, you can use the open source project

`HybridModelBinding` or read the official documentation about custom model binding and implement your own:

1. Custom Model Binding in ASP.NET Core:

<https://adpg.link/65pb>

2. `HybridModelBinding` (open source project on GitHub):

<https://adpg.link/EyKK>

3. Damian Edward's `MinimalApis.Extensions` project on GitHub:

<https://adpg.link/M6zS>

- `ForEvolve.DependencyInjection` is an open source project of mine that adds support for contextual dependency injection and more:
<https://adpg.link/myW8>

[Support](#) [Sign Out](#)

©2022 O'REILLY MEDIA, INC. [TERMS OF SERVICE](#) [PRIVACY POLICY](#)